

Symbolic Model Discovery

T. Tang (with material adapted from Miles Cranmer and Urban Fasel)

Day 4 | Summer School in Hands-on ML 2026

Symbolic model discovery asks for mathematical rule

Search for appropriate mathematical expressions that describe the data

- Prefer rules that remain useful on new cases
- Prefer representations that people can inspect and challenge
- The output is an equation plus its assumptions—not a certificate of truth

Why symbolic models?

Three ambitions: interpretable, generalisable, low complexity

INTERPRETABLE

**Expose variables,
operators and
constants**

**Support expert scrutiny
and scientific reasoning**

GENERALISABLE

**Work on new samples,
conditions or regimes**

**Make the intended
claim testable**

LOW COMPLEXITY

**Evaluate cheaply and
communicate clearly**

**Prefer only the
structure the evidence
supports**

**These are separate ambitions: a short formula can still be unstable,
opaque in context, or scientifically wrong.**

Prediction gives an answer; discovery gives a reusable rule

A predictive model maps measured inputs to an output

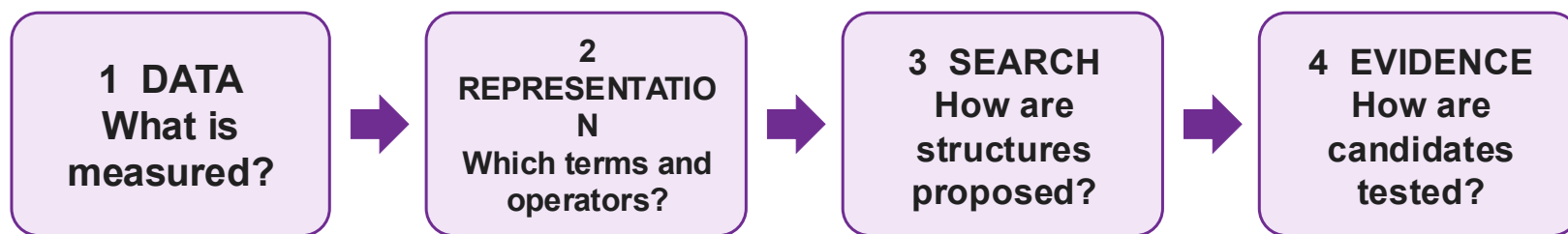
- Equation discovery searches for a compact mathematical relationship
- The equation exposes variables, interactions, nonlinearities and constants
- It can be evaluated, differentiated, integrated and challenged
- Accuracy is necessary; scientific tests establish credibility

Equation discovery searches over model structure

Parameters change numbers inside a chosen form

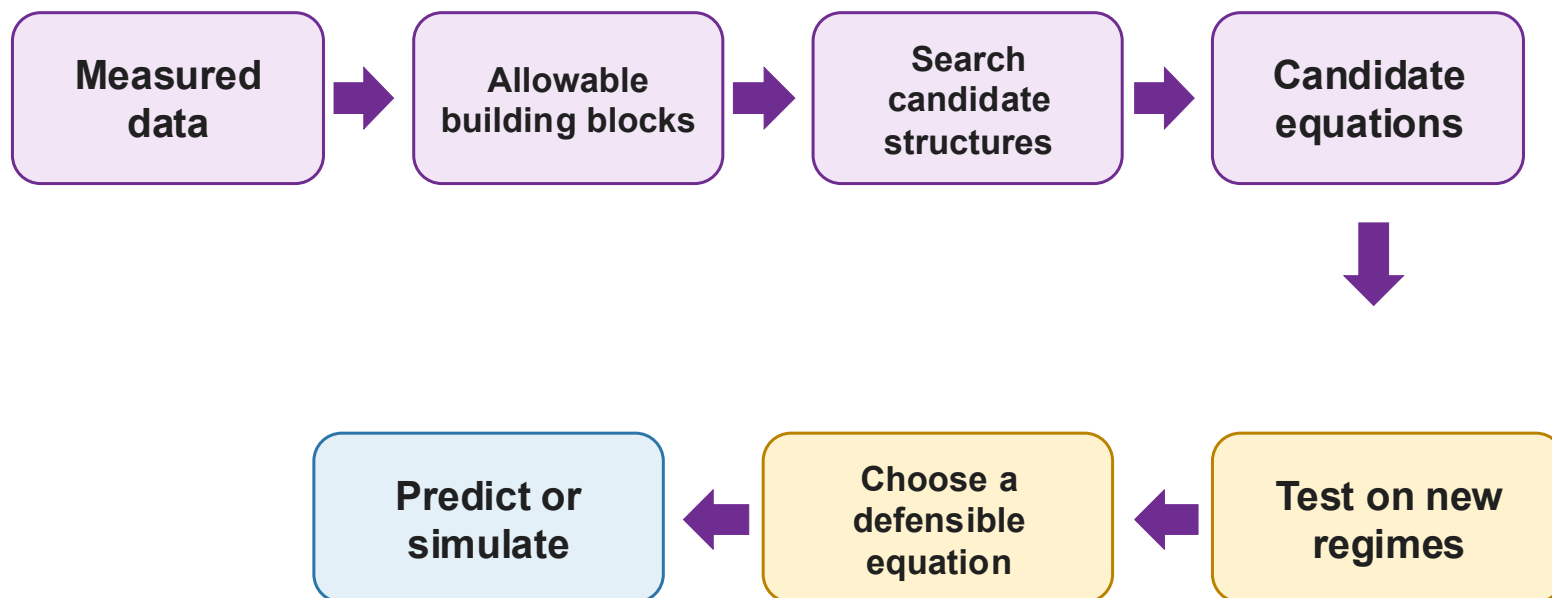
- Structure changes which variables and operators appear
- PySR explores a flexible space of expression trees
- SINDy selects active terms from a fixed candidate library
- Both balance agreement with data against structural simplicity

Every discovery problem has four ingredients



A method cannot recover an equation that its representation makes unreachable.

Equation discovery is a search–test–predict loop



Failed validation sends us back to the data, representation or search.

Symbolic discovery is a landscape—not one algorithm

EXPRESSION TREES

PySR /
SymbolicRegression
.jl
Operon · Eureka
ESR · Brush
AI-Feynman
TuringBot

SPARSE LIBRARIES

PySINDy · PDE-FIND
B-SINDy / E-SINDy
DAHSl · SIDDs
SPIDER · EPDE
SGA-PDE

NEURAL / RL

DSO · DISCOVER
DeepSymReg / EQL
PDE-net · DL-PDE
DLGA-PDE
DySMHO

LOGICAL / PROBABILISTIC

Bayesian Machine
Scientist
AI-Hilbert
AI-Descartes
SciMED · QLattice
Feyn

Classify a method by its search space, training regime and learning target—not its package name.

Methods differ mainly in how they search structure

1 EXPRESSION TREE + EVOLUTION

Propose a formula tree
Mutate or cross subtrees
Optimise constants
Retain accuracy–
complexity trade-offs

PySR · Operon · Eureqa

2 LIBRARY + SPARSE REGRESSION

Evaluate chosen
candidate terms
Fit coefficients
Remove weak terms
Refit the surviving support

SINDy · PDE-FIND ·
SPIDER

3 NEURAL / RL / GENERATIVE

Learn how to propose
structures
Use gradients, policies or
sequence models
Reward fit and simplicity

DSO · DISCOVER ·
DeepSymReg

This lecture focuses on regimes 1 and 2 because their backends expose complementary ideas clearly.

Before choosing software, choose the mathematical target

DIRECT SOLUTION OR MAP

Learn $y = f(X)$

Input row $\rightarrow$ output value

Prediction evaluates the learned expression once for each new row

Examples: PySR, QLattice, Operon



DIFFERENTIAL OPERATOR

Learn $dx/dt = f(x, u)$

State $\rightarrow$ instantaneous rate of change

Prediction repeatedly integrates the learned law from an initial state

Examples: PySINDy, PDE-FIND, SPIDER

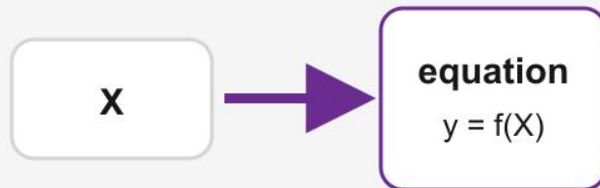
Time-series data can support either target—but the equations, preprocessing and validation question are different.

One question, two discovery targets

SYMBOLIC REGRESSION

Static relation

Paired observations: features X and target y



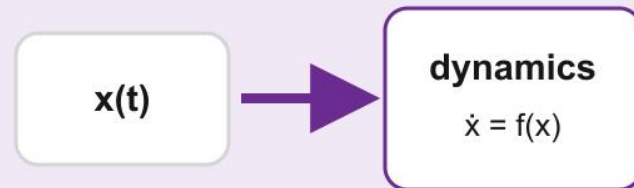
Best when you want:

- a compact predictive law
- interpretable feature interactions

SINDY

Governing dynamics

Time-ordered state trajectories $x(t)$

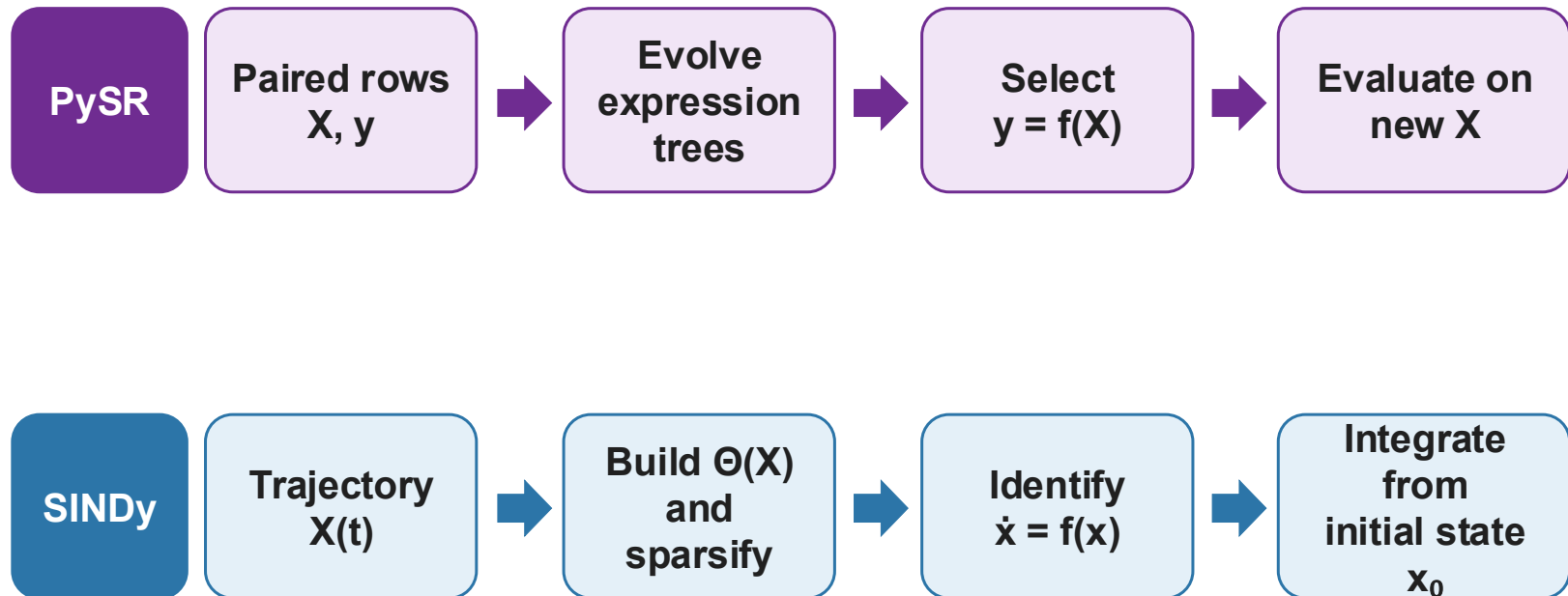


Best when you want:

- a sparse ODE or PDE
- simulation from new initial states

Choose the target before choosing the package.

PySR and SINDy take different routes to a prediction



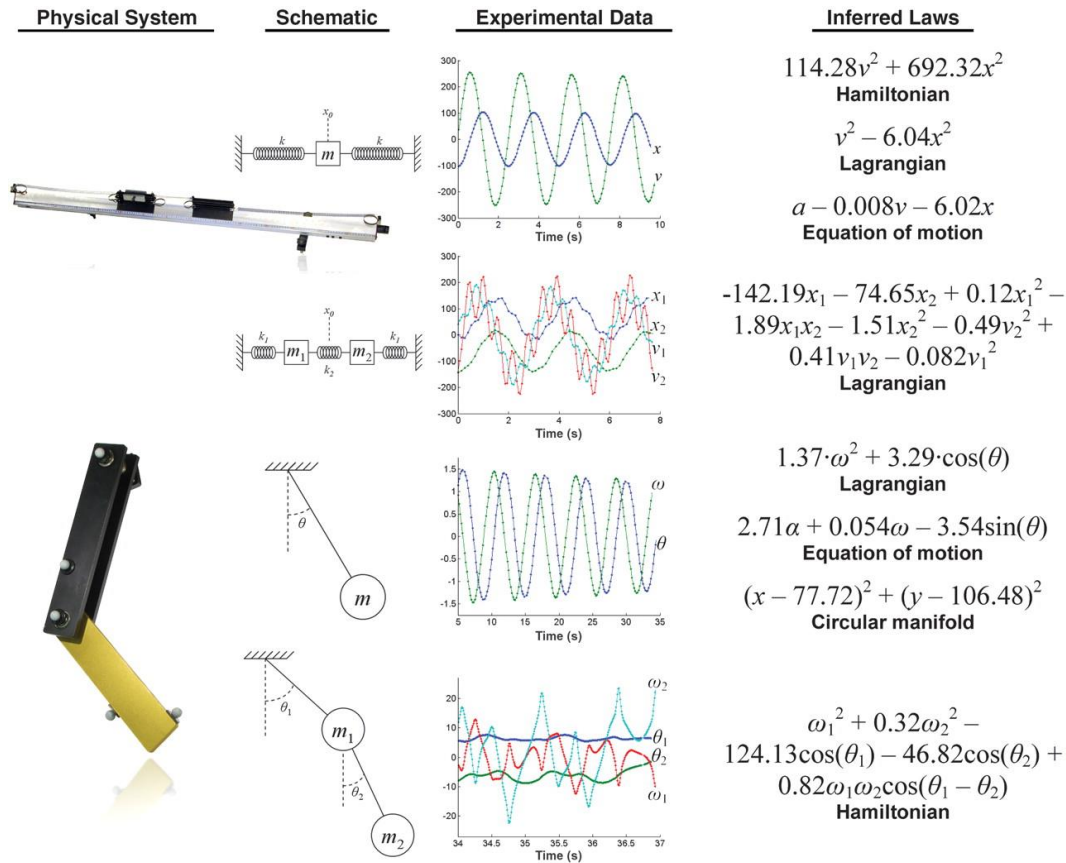
Interpretability is a workflow— not a label

Use a representation people can read

- Encode scientific constraints before searching
- Check whether structure survives new data and new runs
- Test behaviour outside the fitted samples
- Keep the assumptions open to expert challenge

Use cases | What equations unlock

Scientific discovery | Rediscover laws from motion

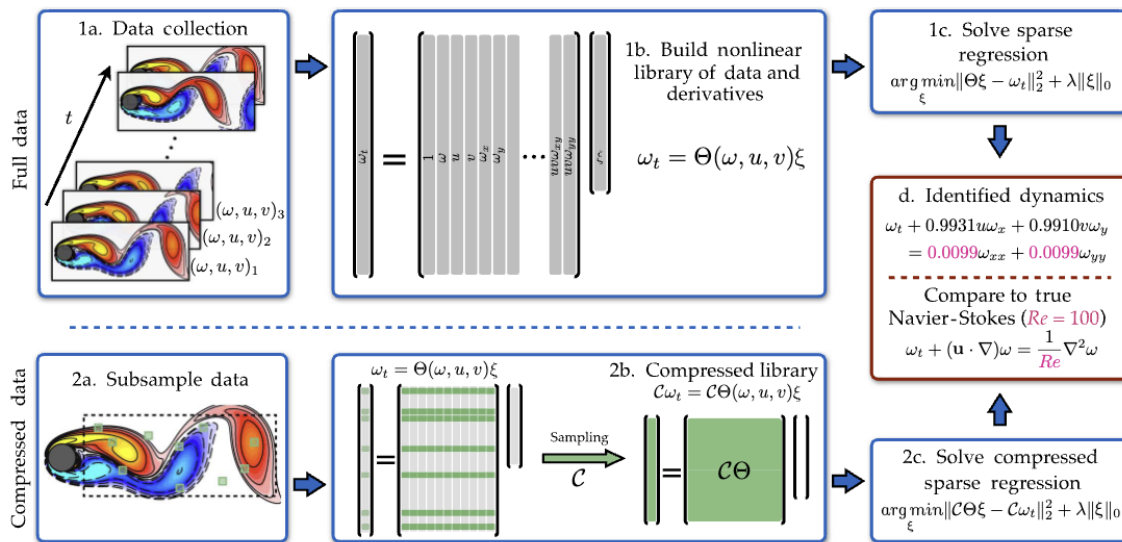


**Motion-tracking data →
free-form equations of
motion**

**The equation is judged
by whether it
reconstructs several
observed dynamical
relationships—not by
one fitted curve.**

Adapted from the Day 4 workshop
welcome deck

Fluid mechanics | Discover a PDE from field data



Fields $u(x,t) \rightarrow$ candidate derivatives and nonlinear terms $\rightarrow$ sparse PDE

Here the target is a differential operator, not a direct future snapshot.

Control and robotics | Identify a model that can act

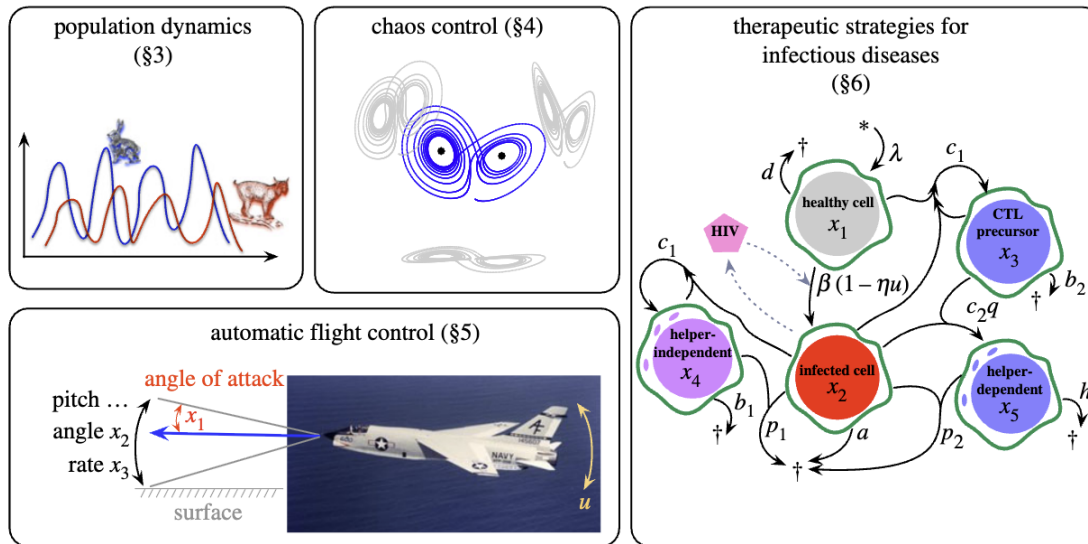


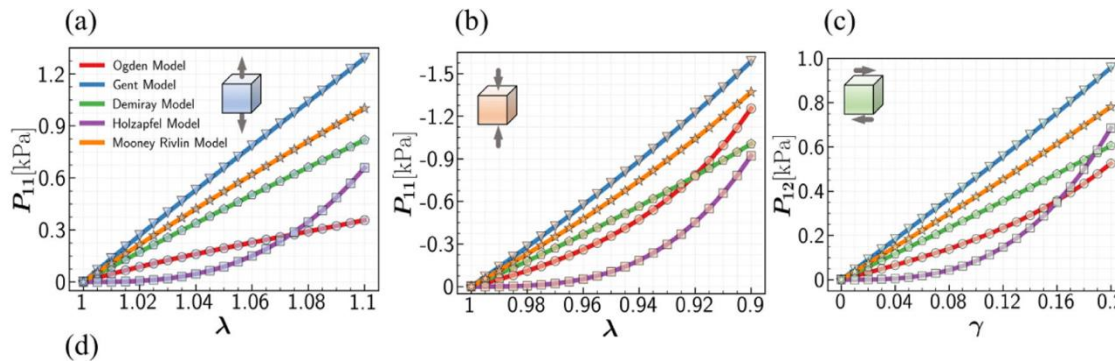
Figure 1. Applications of SINDY-MPC investigated in this work. (Online version in colour.)

**Sparse dynamics +
model-predictive control**

**A useful discovered
model must roll forward
quickly and accurately
enough to choose an
action under limited data.**

Adapted from the Day 4 workshop
welcome deck

Healthcare and materials | Discover constitutive laws



Ogden model: $\Psi = 0.0102(\lambda_1^{-18} + \lambda_2^{-18} + \lambda_3^{-18} - 3)$

Gent model: $\Psi = -1.899 \ln(1 - 1.201[I_1 - 3])$

Demiray model: $\Psi = 1.661(\exp(0.8802[I_1 - 3]) - 1)$

Holzapfel model: $\Psi = 5.602(\exp(3.001[I_1 - 3]^2) - 1)$

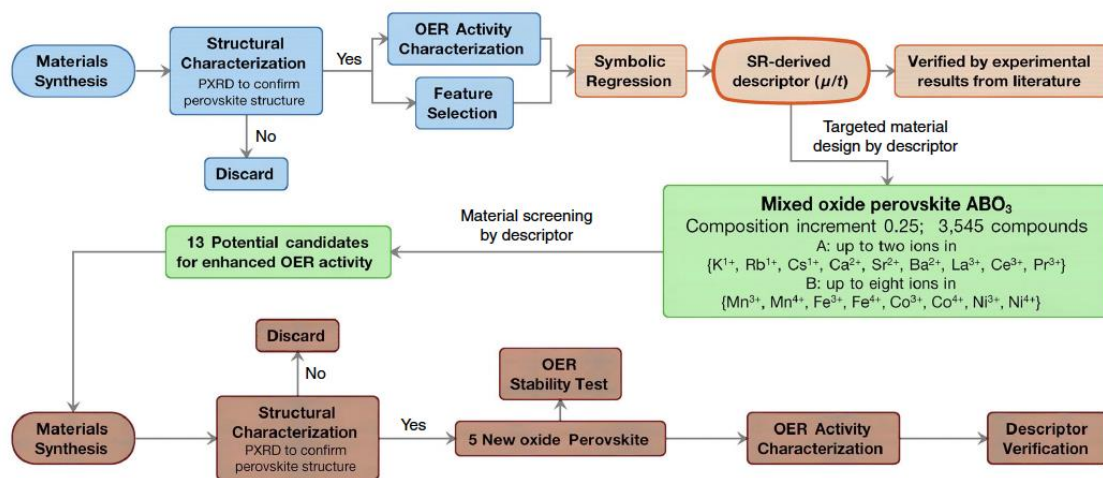
Mooney Rivlin model: $\Psi = 0.8702[I_1 - 3] + 0.8593[I_1 - 3]^2 + 0.9798[I_2 - 3] + 0.4303[I_2 - 3]^2$

**Stress–strain
measurements →
compact constitutive
equations**

**Physics consistency and
asymmetric nonlinear
response matter
alongside numerical
accuracy.**

Adapted from the Day 4 workshop
welcome deck

Chemistry | Turn a descriptor into an experimental decision

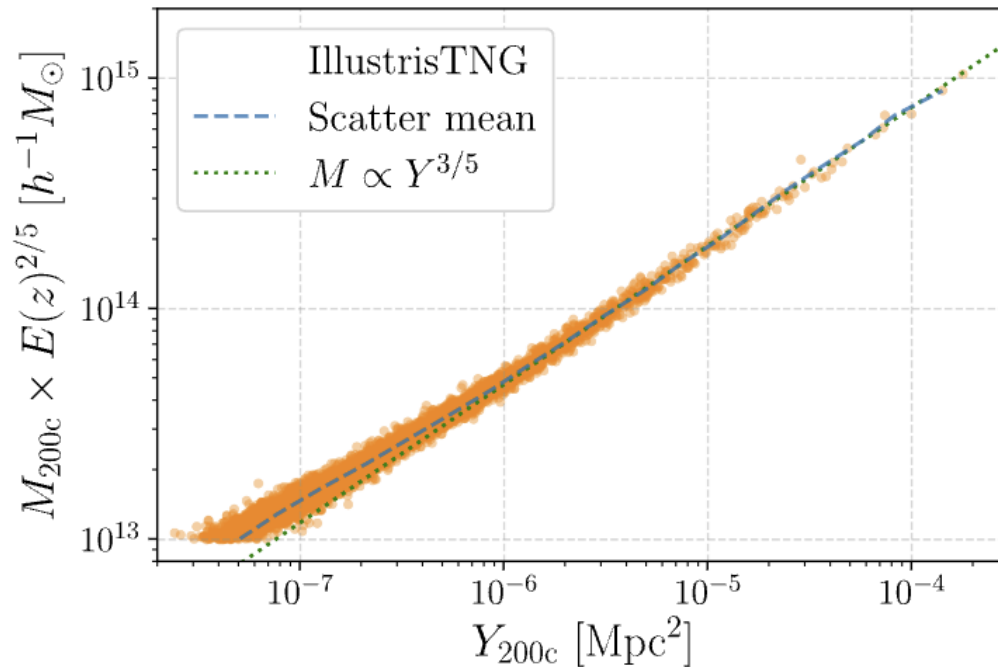


Materials database →
simple descriptor μ/t →
synthesis and
verification

The strongest evidence
is experimental success
after the equation is
proposed.

Adapted from the Day 4 workshop
welcome deck

Astronomy | Improve a scaling law without losing meaning

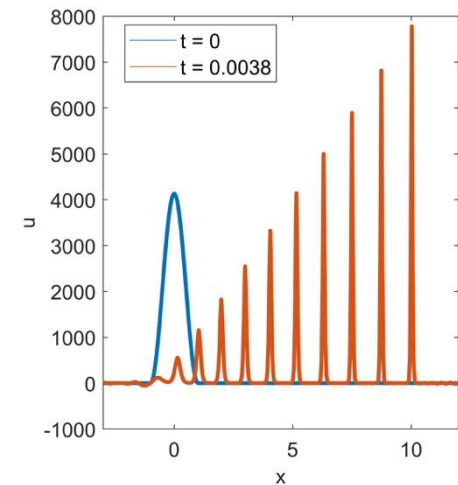
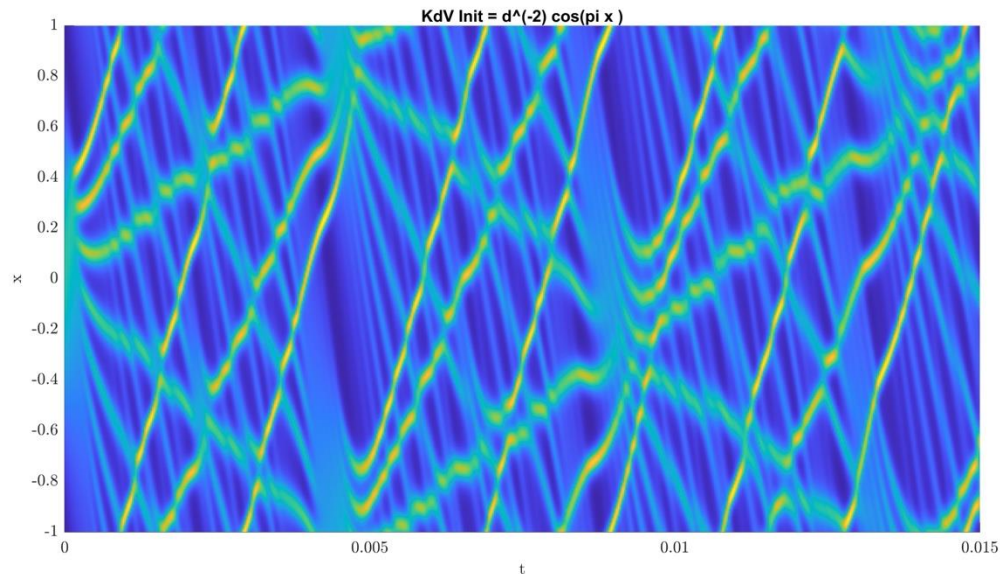


Simulation catalogue → amended mass proxy

The added concentration term reduced scatter by roughly 20–30% and remained interpretable to domain scientists.

Adapted from the Day 4 workshop welcome deck

Applied mathematics | Discover conserved predictive structure



**KdV field data → conserved and empirically conserved quantities → predict
each outgoing soliton's shape and height**

1. Symbolic regression with PySR

Regression chooses parameters; PySR also chooses form

A standard model fixes the functional form

- Symbolic regression searches the form and its constants
- Score candidates on empirical error and expression complexity
- Return a set of trade-offs—not one unquestionable answer
- Example: $2.54 \cos(x_3) + x_0^2 - 2$

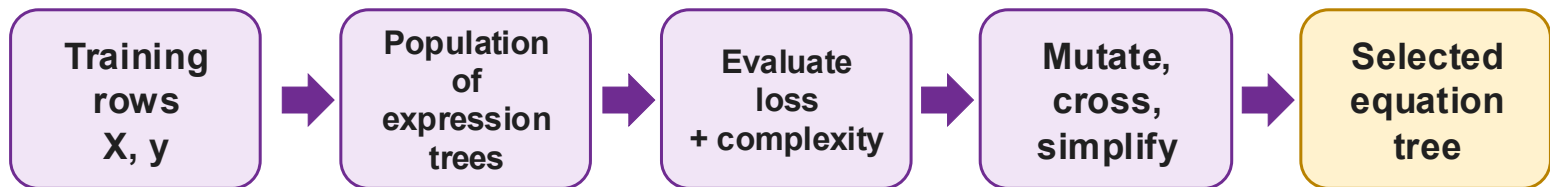
PySR starts with a population of expression trees

Leaves are variables or numerical constants

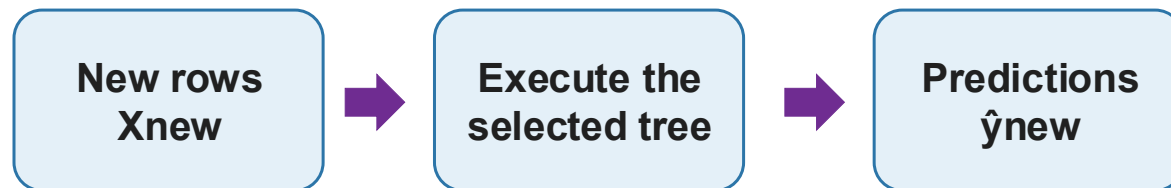
- Internal nodes are allowed unary or binary operators
- maxsize, maxdepth and nesting rules reject invalid trees
- Every valid tree is executable: $X \rightarrow$ candidate predictions $\hat{y}$
- The search is discrete in structure and continuous in constants

Prediction executes PySR's learned tree

**DURING
fit()**

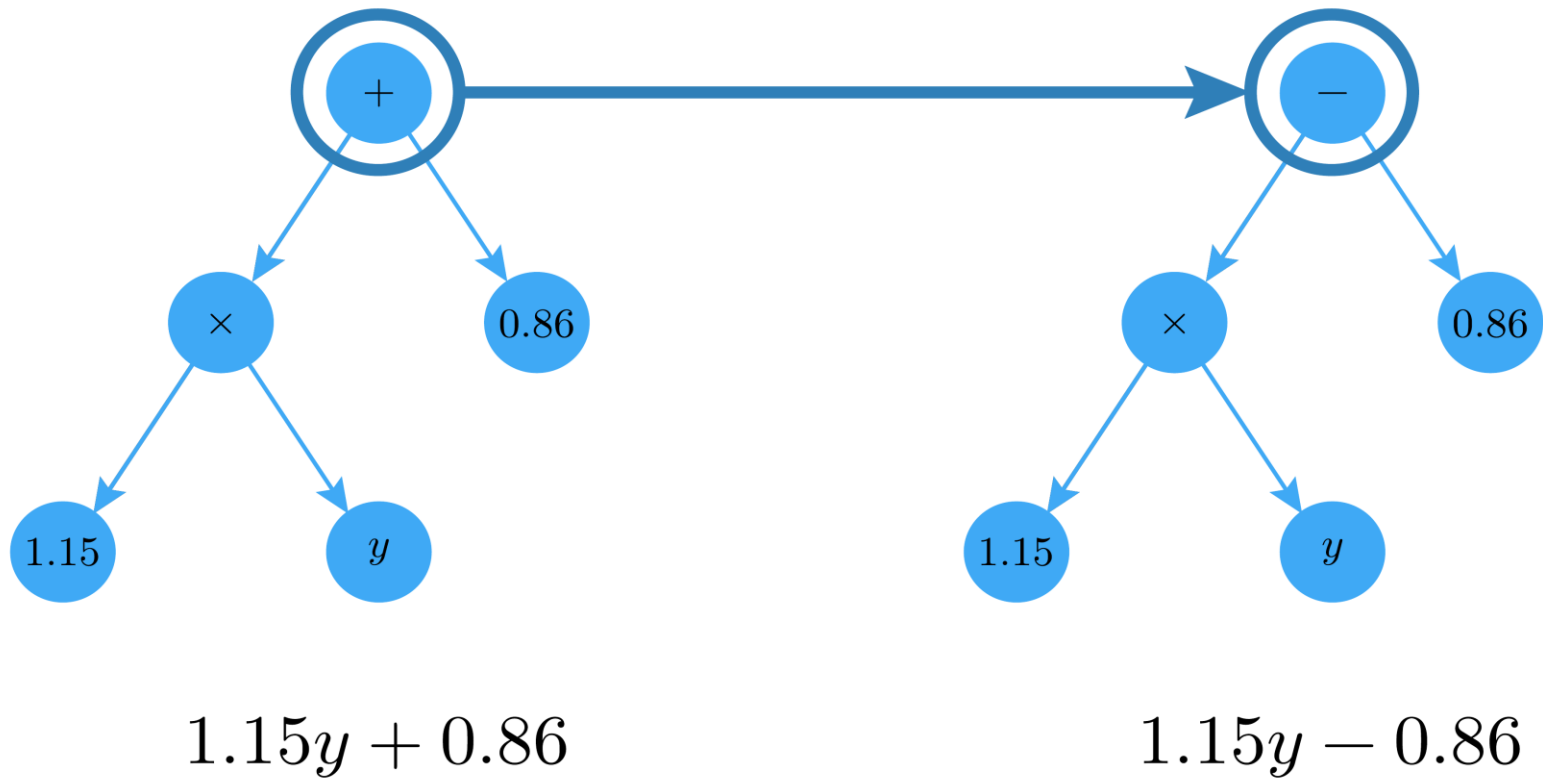


**DURING
predict()**



Prediction evaluates the learned formula; it does not restart the search.

A mutation changes one node—and therefore the formula

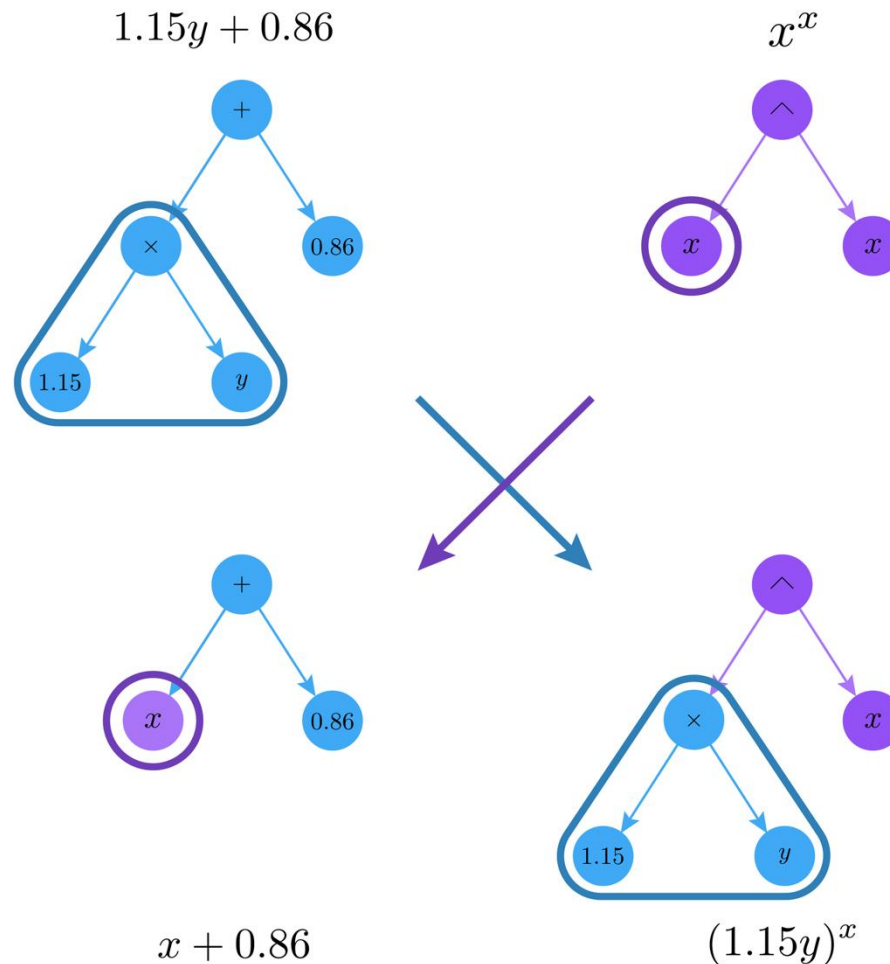


Mutation makes a local structural proposal

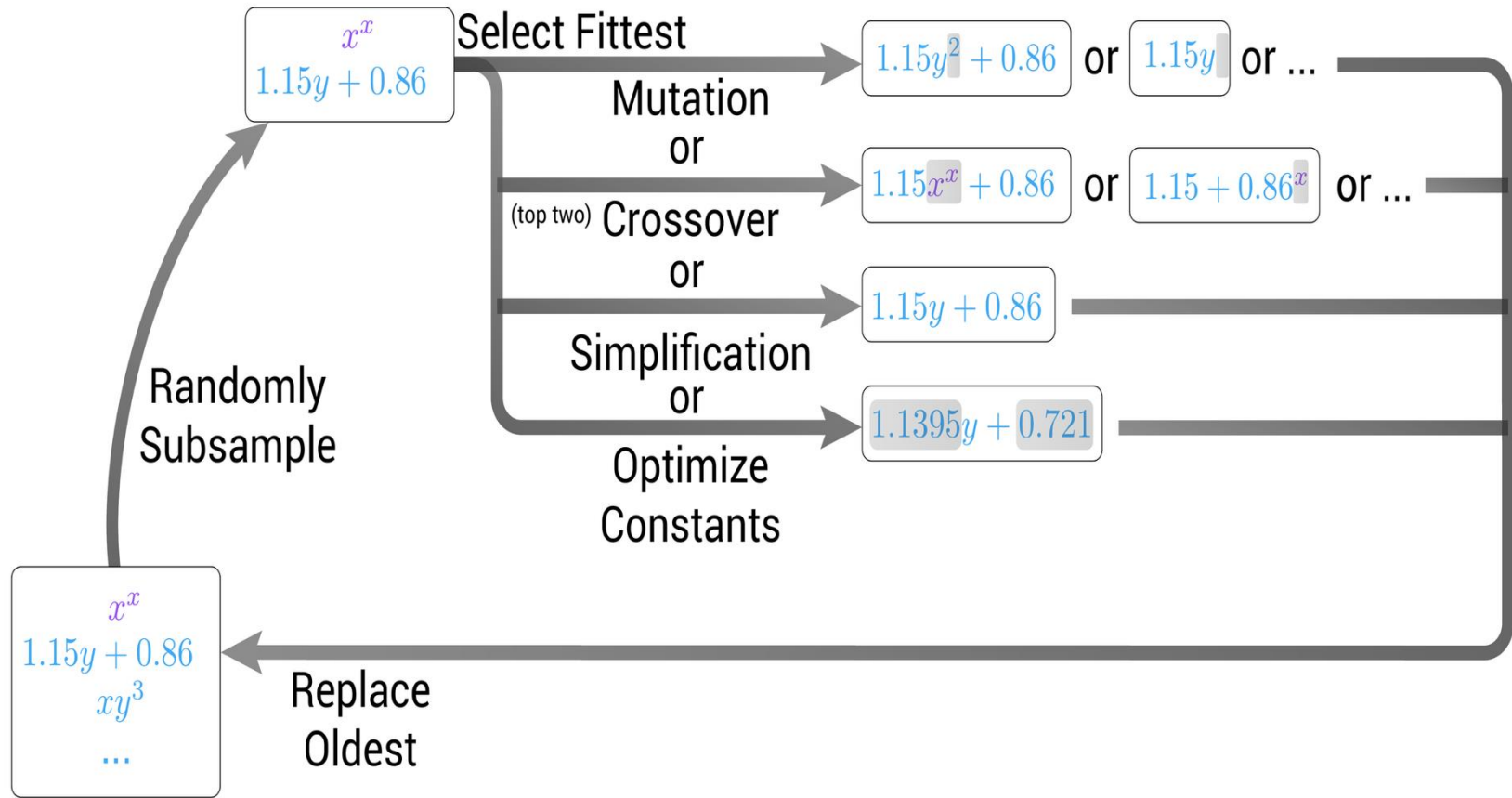
Choose a parent by tournament selection

- Copy its tree; then perturb one part
- Change an operator or numerical constant
- Insert a node, delete a subtree, simplify—or start again
- Reject proposals that violate hard structural constraints

Crossover swaps complete subtrees between parents



One evolutionary cycle breeds, simplifies and optimises

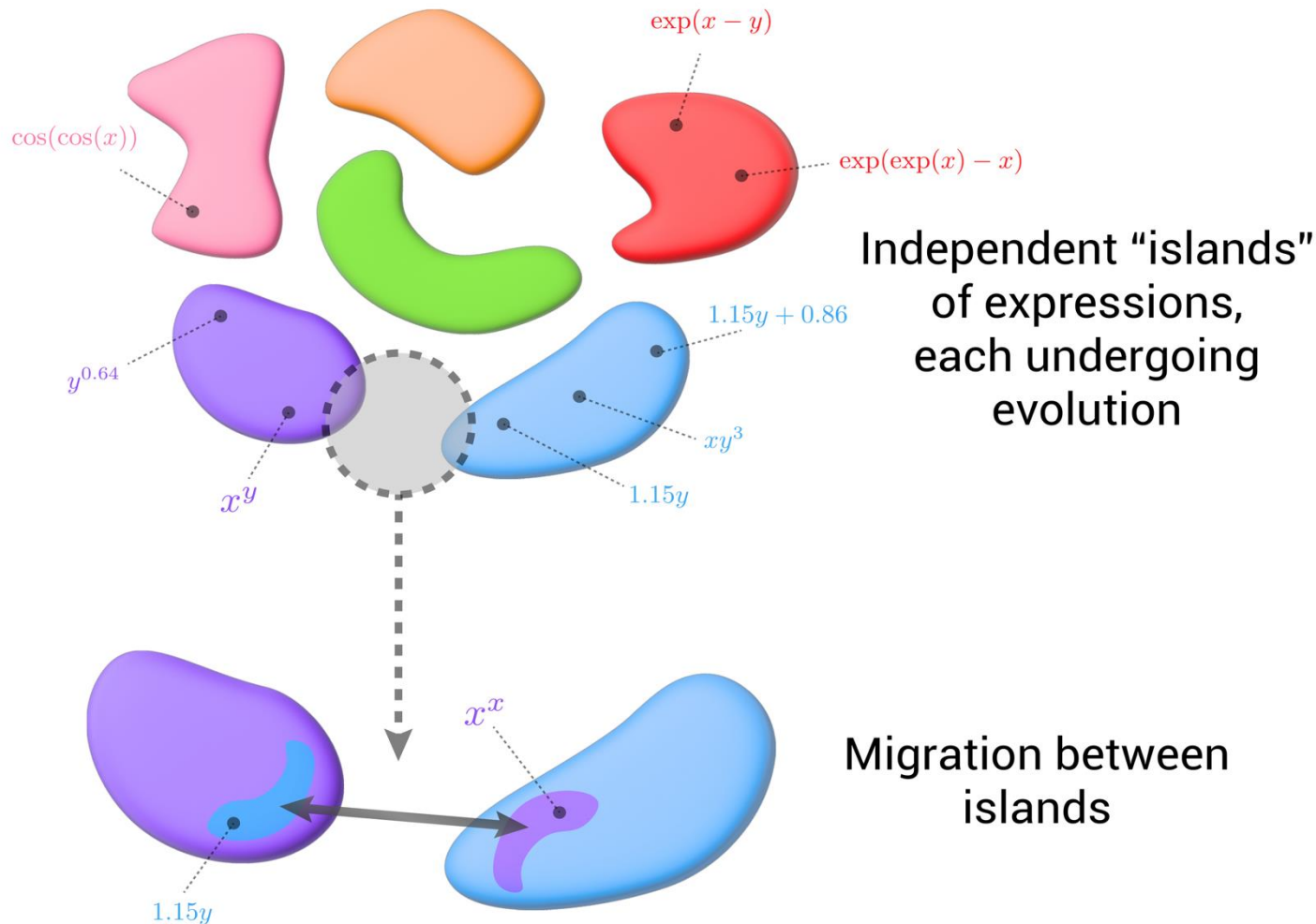


Constant optimisation is nested inside tree search

Hold the expression topology fixed

- Treat numerical leaves as continuous parameters
- Use a local optimiser such as BFGS or Nelder–Mead
- Update constants to reduce loss for that structure
- Return the improved equation to the evolutionary search

Independent populations explore equation families



The Python call launches the backend search

```
from pysr import PySRRegressor

model = PySRRegressor(
    niterations=200,
    binary_operators=["+", "*", "/"],
    unary_operators=["cos"],
    maxsize=20,
)

model.fit(X_train, y_train)

model.equations_

model.predict(X_test)
```

WORKFLOW

1 Choose operators

2 Search
error × complexity

3 Inspect the set
not only the top row

4 Validate
held-out regimes

The operator set is a scientific prior

Start with $+$, $-$, $\times$ and $\div$

- Add $\sin$, $\exp$ or $\log$ only with a scientific reason
- Constrain invalid nesting and denominator structure
- Use units or dimensional analysis to reject nonsense
- Custom losses can encode what 'good' means

Constraints define the grammar of the search

maxsize caps the total expression tree

- Operator costs change which forms are preferred
- Nested constraints block pathological combinations
- Dimensional analysis excludes inadmissible equations
- A smaller grammar searches faster and argues more clearly

Noise changes both the loss and the search

Heteroscedastic noise: use weights proportional to $1/\sigma^2$

- Denoising can reveal structure—but can also invent smoothness
- Split by operating regime, not only random rows
- Refit constants on held-out data
- Repeat the search: instability is evidence

PySR | Controlling overfitting

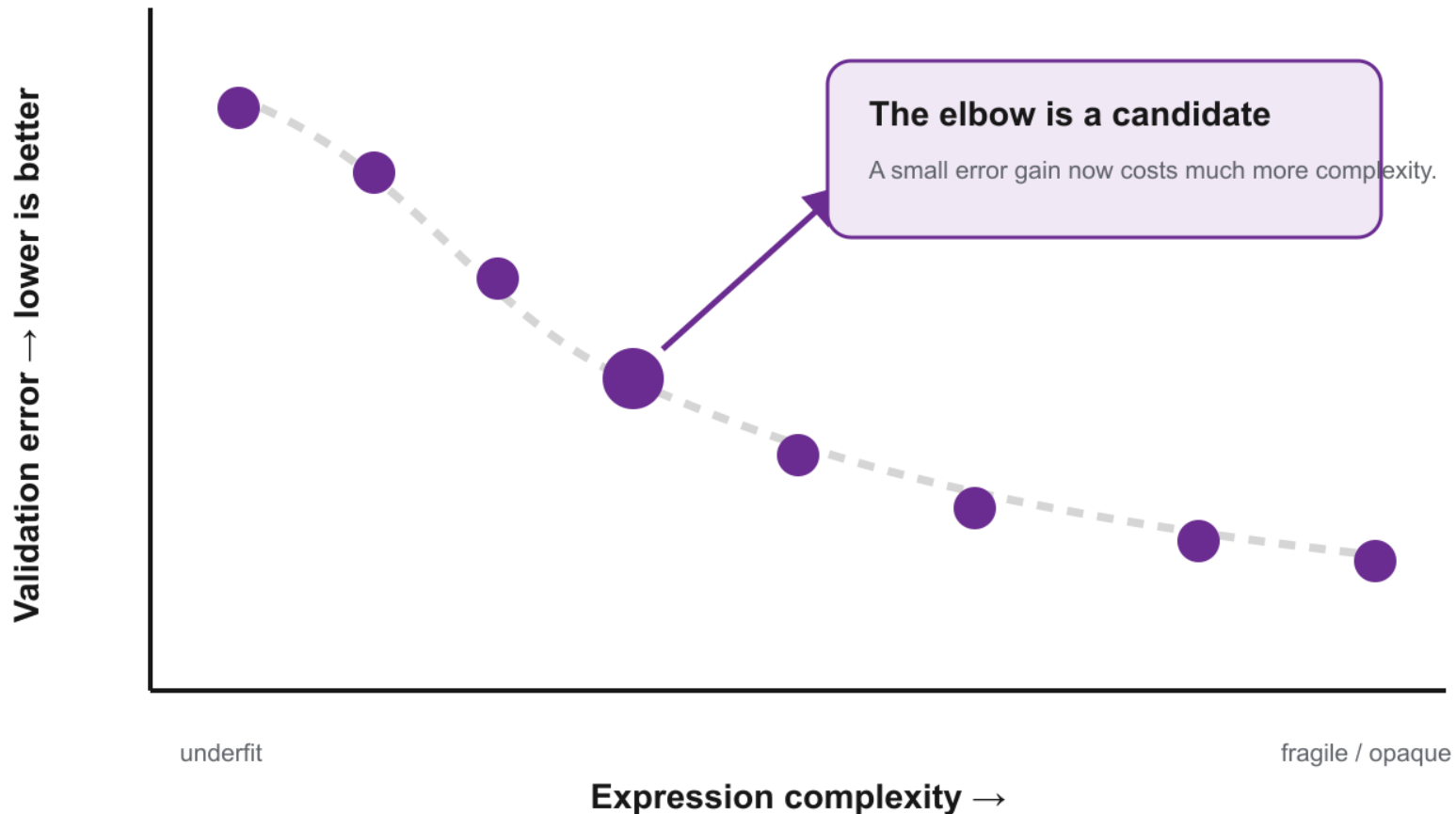
PySR reduces overfitting in layers—it does not eliminate it



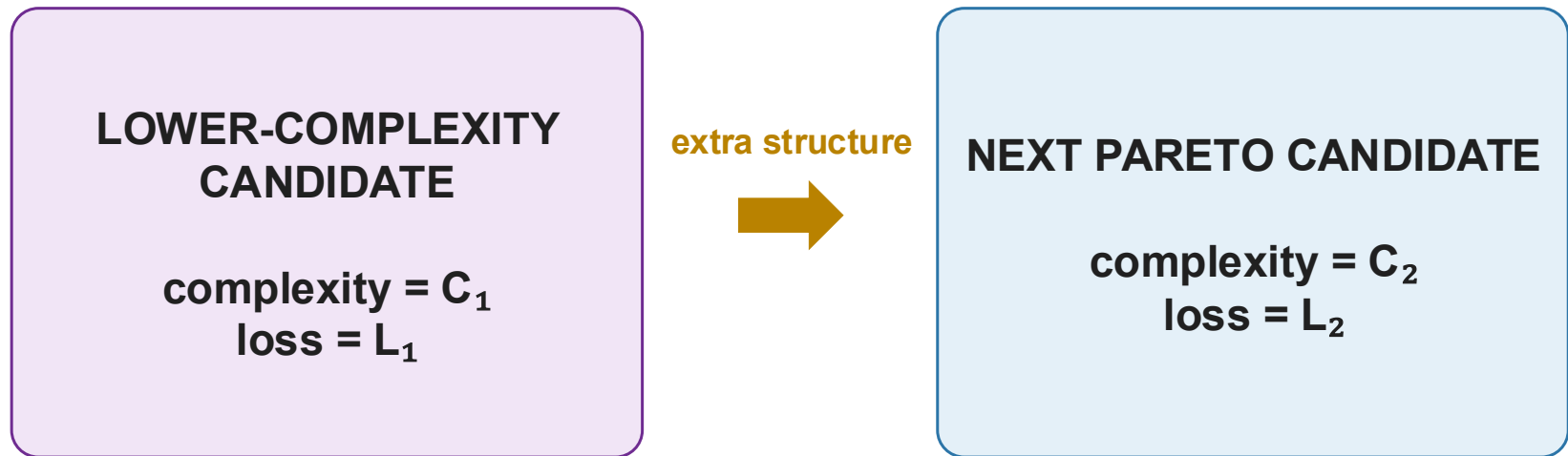
Training loss still drives the search. Only unseen data can test whether the selected equation generalises.

Read the Pareto front—not only the top row

MODEL SELECTION



PySR asks how much loss is bought by extra complexity



$$\text{score} \approx - \Delta \log(\text{loss}) / \Delta \text{complexity}$$

High score: a large relative loss reduction for little added complexity. Near zero: complexity buys almost nothing.

Worked example: the middle equation earns its extra complexity

CANDIDATE	COMPLEXITY	LOSS	LOSS-GAIN SCORE
A	5	0.100	baseline
B	6	0.060	0.51 ← strongest gain
C	9	0.055	0.029

B adds one unit of complexity and cuts loss by 40%; C adds three more units for only another 8% reduction.

PySR 'best': among equations with loss $\leq 1.5 \times$ minimum loss, select the highest score. This is still selection on fitted loss.

Use PySR's score to shortlist; use held-out data to decide



Split by the scientific claim: new operating range, subject, instrument or time block—not merely neighbouring random rows.

2. Sparse dynamics with SINDy

From trajectories to governing equations

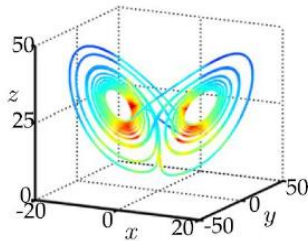
Data



Dynamics (assumptions)

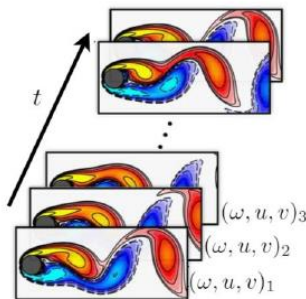


Model structure
& coefficients



$$\frac{d}{dt}\mathbf{x} = \mathbf{f}(\mathbf{x})$$

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z\end{aligned}$$



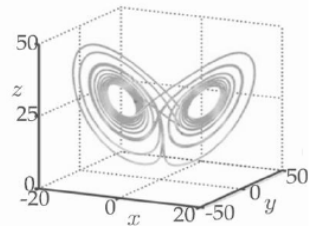
$$\frac{\partial \mathbf{u}}{\partial t} = \mathbf{N}(\mathbf{u})$$

$$\omega_t + (\mathbf{u} \cdot \nabla)\omega = \frac{1}{Re} \nabla^2 \omega$$

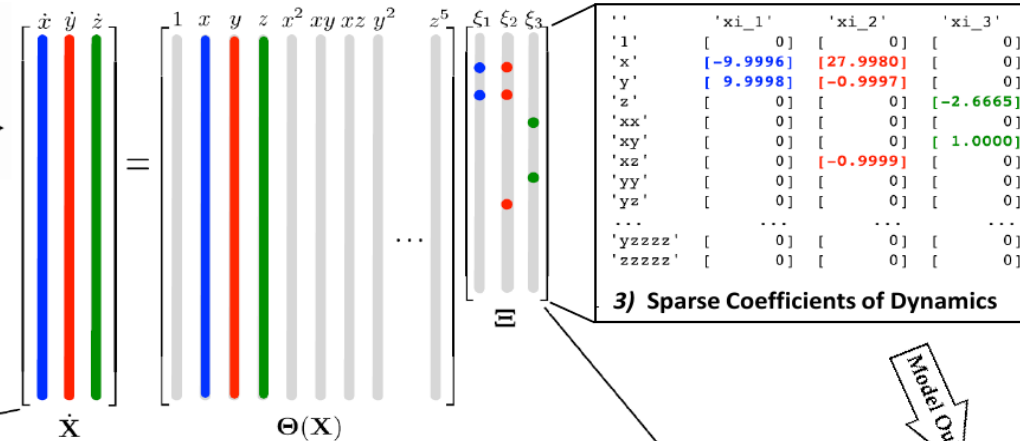
SINDy in four objects: X , $\dot{X}$, $\Theta(X)$, Ξ

1) Lorenz System Data

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z.\end{aligned}$$

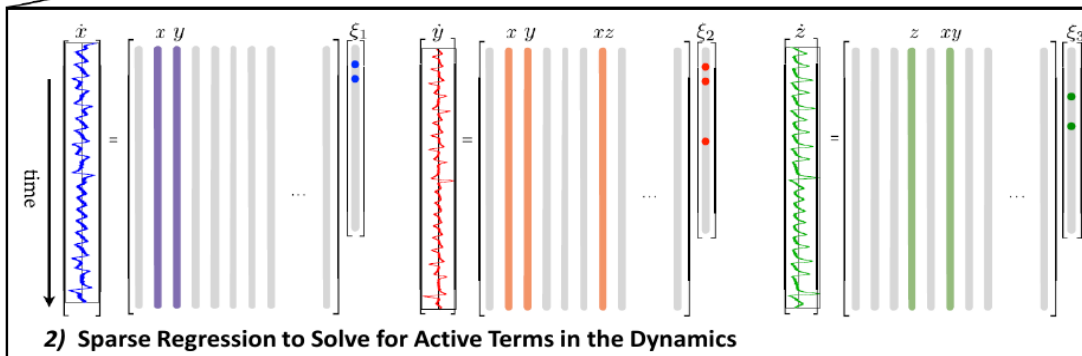


Data In

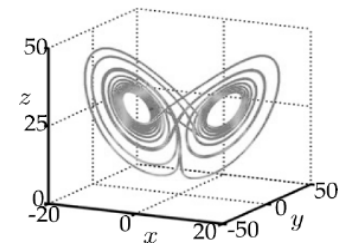


3) Sparse Coefficients of Dynamics

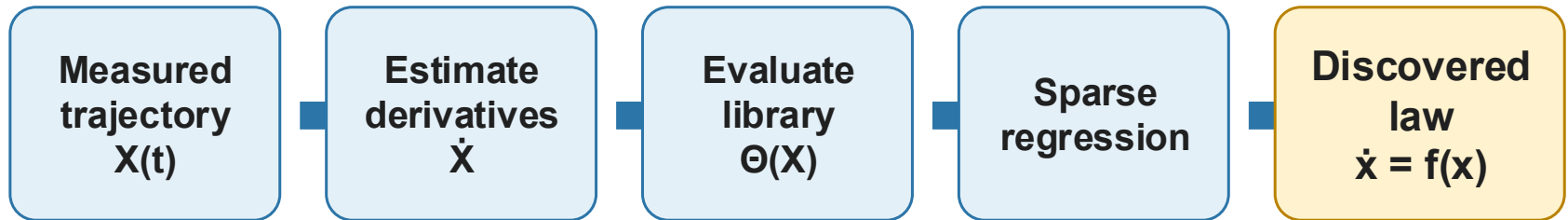
Model Out



4) Identified SINDy model prediction



SINDy learns the right-hand side of a differential equation



The output of `fit()` is a right-hand-side function—not a stored future trajectory.

SINDy works under testable assumptions

Measured states approximately close the system

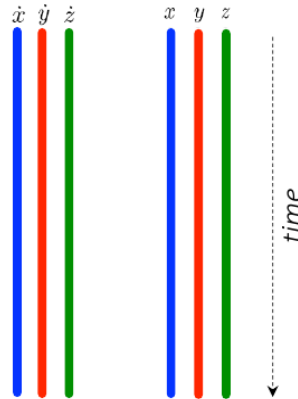
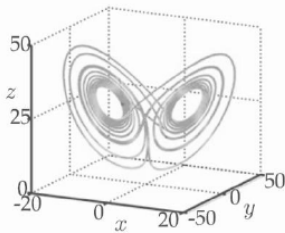
- Only a few library terms are active
- Derivatives—or weak integrals—can be estimated
- Sampling resolves the relevant time scales
- Forcing and control inputs are modelled, not hidden

Differentiation is the first risk

1) Lorenz System Data

$$\begin{aligned}\dot{x} &= \sigma(y - x) \\ \dot{y} &= x(\rho - z) - y \\ \dot{z} &= xy - \beta z.\end{aligned}$$

Data In



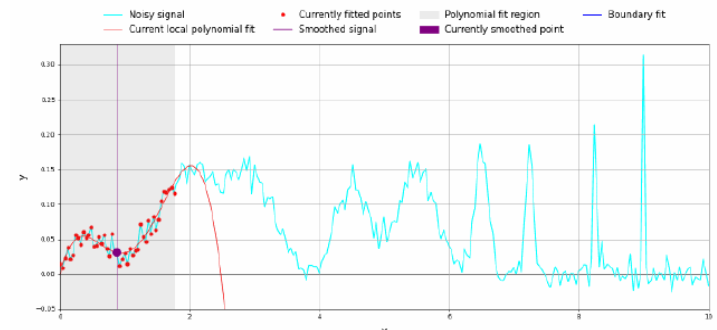
- Different orders of accuracy
- e.g. 1st derivative using central difference
 - 2nd order accuracy: $\dot{x}_t \approx \frac{1}{\Delta t} \left(\frac{1}{2} x_{t+1} - \frac{1}{2} x_{t-1} \right)$
 - 4th order accuracy: $\dot{x}_t \approx \frac{1}{\Delta t} \left(-\frac{1}{12} x_{t+2} + \frac{2}{3} x_{t+1} - \frac{2}{3} x_{t-1} + \frac{1}{12} x_{t-2} \right)$

Smoothed Finite Difference → noisy data

- Finite Difference on smoothed/filtered data
 - e.g. Savitzky Golay: least-squares fit of a polynomial to the data
 - applied to the data before differentiation

Polynomial (Savitzky Golay) derivative → noisy data

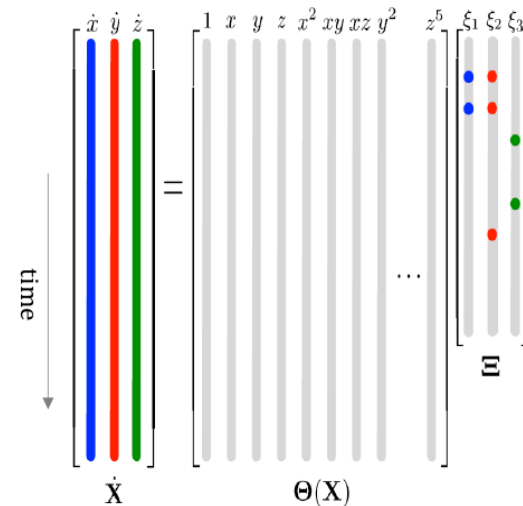
- Analytically compute the derivative of the fitted polynomial



The feature library is the hypothesis space

General approach to selecting the library

- Start with linear terms → DMD model
 - Check accuracy
 - error reconstructing $\dot{\mathbf{X}}$
 - model prediction error
- Increase order
 - Add quadratic, then higher order polynomials
- Trigonometric functions
- Generally: try small, isolated libraries first



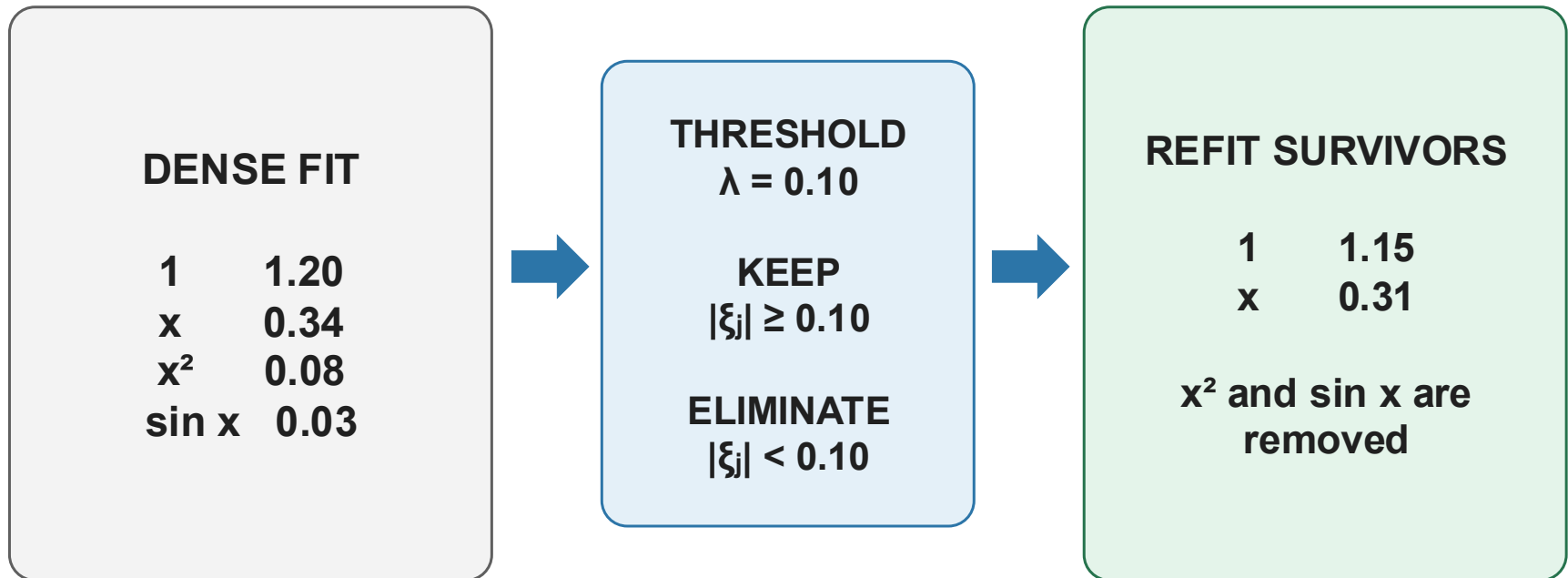
SINDy makes nonlinear discovery linear in coefficients

Choose candidates $\theta_j(x)$: 1, x , y , x^2 , xy , $\sin(x)$, ...

- Evaluate every θ_j at every time sample
- Stack the evaluated candidates as columns of $\Theta(X)$
- The functions may be nonlinear in the measured state
- But $\dot{x} = \Theta(X)\xi$ is linear in the unknown coefficients ξ

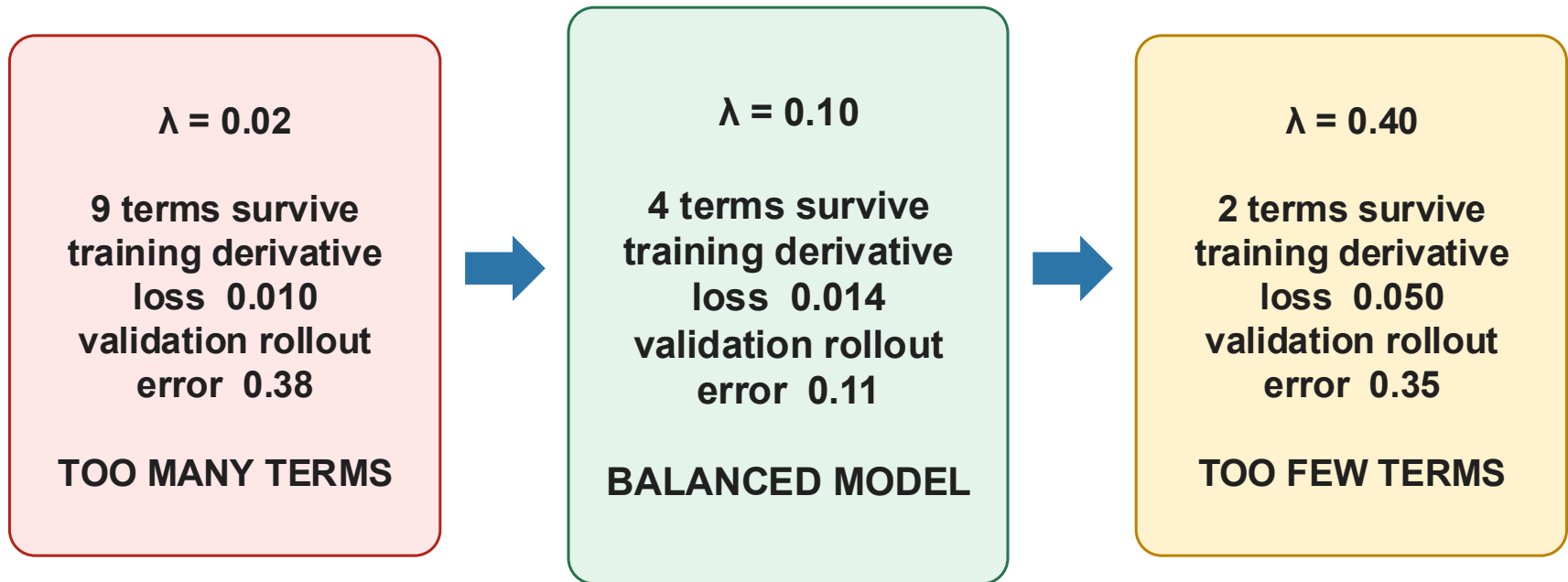
SINDy | Choosing how many terms survive

SINDy's threshold decides which coefficients are eliminated



λ does not directly say 'keep two terms'. It sets a magnitude cutoff; the retained-term count is the consequence. Scaling changes that consequence.

Choose λ by tracing retained terms against validation behaviour



higher λ → more coefficients eliminated → fewer terms

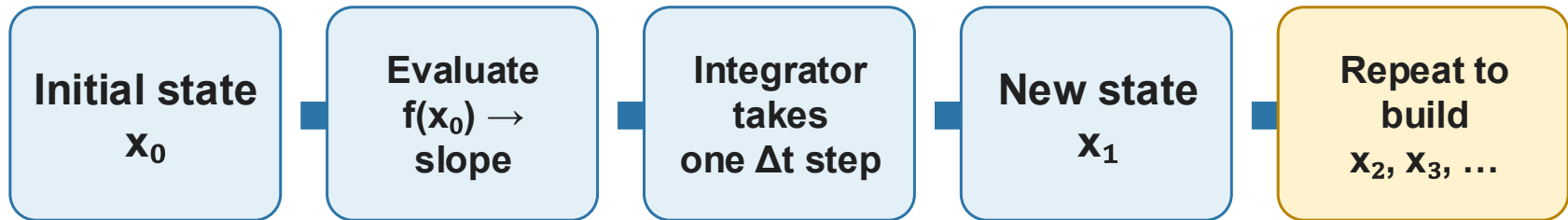
Choose the threshold using held-out rollout error and support stability—not the smallest training loss. Values shown are schematic.

Threshold, remove, refit—then repeat

Start from a dense coefficient estimate

- Set coefficients with $|\xi_j| < \lambda$ to zero
- Remove those terms from the active support
- Refit the remaining coefficients
- Repeat until support stops changing; λ controls sparsity

SINDy predicts by repeatedly integrating the discovered law



The sequence $x_0, x_1, x_2, \dots$ is the predicted trajectory.

Small slope errors can accumulate at every integration step.

PySINDy exposes the three modelling choices

```
import pysindy as ps

model = ps.SINDy(
    differentiation_method=ps.SmoothedFiniteDifference(),
    feature_library=ps.PolynomialLibrary(degree=3),
    optimizer=ps.STLSQ(threshold=0.1),
)

model.fit(x_train, t=dt)

model.print()

x_sim = model.simulate(x0, t_test)
```

THREE CHOICES

Differentiate

estimate $\dot{X}$ without amplifying noise

Build $\Theta(X)$

encode plausible mechanisms

Sparse fit

select terms and coefficients

Validate dynamics by rolling them forward

Derivative fit is necessary but insufficient

- Integrate from unseen initial conditions
- Compare trajectories, attractors and conserved quantities
- Test across sampling rates and noise levels
- Stable support and coefficients matter more than tiny training error

3. Robust discovery and model selection

Select complexity with out-of-sample evidence

Model selection is **not simply about reducing error**, but about producing a model that has high degree of **interpretability**, **generalization**, and **predictive capabilities**.

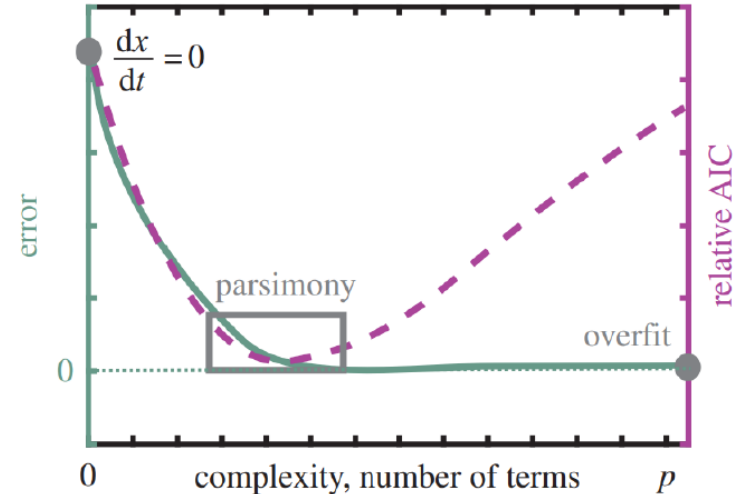
→ selecting **accurate** and **sparse** model

Model selection methods

- Cross validation
- Prediction error & sparsity
- Stability selection

Tutorials

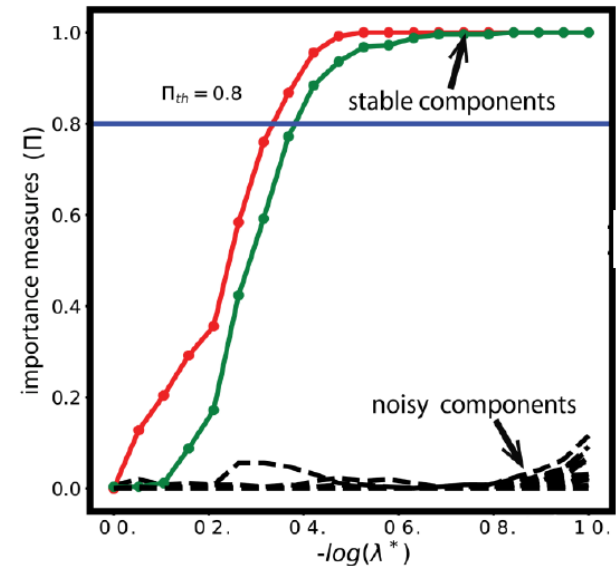
- Compare methods on Lorenz system data



Stable terms survive perturbation

Calculate the stability (importance) of each coefficient over the regularization path λ

1. Generate B random subsamples (size $N/2$) of the library and derivative data without replacement
2. Compute B SINDy models over regularization path λ
3. Calculate the λ -dependent stability (or importance) Π_k^λ
 - **Stability**: probability of a coefficient to be non-zero
 - Find stable support: $\Pi_k^\lambda > \pi_{th} = 0.8$
 - e.g. coefficient is non-zero in 80% of the models



PySR evolves trees; SINDy sparsifies a library

PySR		SINDy / PySINDy	
TARGET	$y = f(X)$	TARGET	$\dot{x} = f(x)$ or PDE
DATA	paired rows	DATA	time-ordered trajectories
SEARCH	expression trees	SEARCH	sparse terms in $\Theta(X)$
MAIN RISK	operator-space bias	MAIN RISK	derivatives + library bias
VALIDATE	held-out regimes	VALIDATE	simulate from unseen states